

CHAPTER 3

Memory Consistency

Motivation and Sequential Consistency

This chapter delves into memory consistency models (a.k.a. memory models) that define the behavior of shared memory systems for programmers and implementors. These models define correctness so that programmers know what to expect and implementors know what to provide. We first motivate the need to define memory behavior (Section 3.1), say what a memory consistency model should do (Section 3.2), and compare and contrast consistency and coherence (Section 3.3).

We then explore the (relatively) intuitive model of sequential consistency (SC). SC is important because it is what many programmers expect of shared memory and provides a foundation for understanding the more relaxed (weak) memory consistency models presented in the next two chapters. We first present the basic idea of SC (Section 3.4) and present a formalism of it that we will also use in subsequent chapters (Section 3.5). We then discuss implementations of SC, starting with naive implementations that serve as operational models (Section 3.6), a basic implementation of SC with cache coherence (Section 3.7), more optimized implementations of SC with cache coherence (Section 3.8), and the implementation of atomic operations (Section 3.9). We conclude our discussion of SC by providing a MIPS R10000 case study (Section 3.10) and pointing to some further reading (Section 3.11).

3.1 PROBLEMS WITH SHARED MEMORY BEHAVIOR

To see why shared memory behavior must be defined, consider the example execution of two cores¹ depicted in Table 3.1. (This example, as is the case for all examples in this chapter, assumes that the initial values of all variables are zero.) Most programmers would expect that core C2's register r2 should get the value NEW. Nevertheless, r2 can be 0 in some of today's computer systems.

Hardware can make r2 get the value 0 by reordering core C1's stores S1 and S2. Locally (i.e., if we look only at C1's execution and do not consider interactions with other threads), this reordering seems correct because S1 and S2 access different addresses. The sidebar on page 18

¹Let “core” refer to software's view of a core, which may be an actual core or a thread context of a multithreaded core.

18 3. MEMORY CONSISTENCY MOTIVATION AND SEQUENTIAL CONSISTENCY

Table 3.1: Should r2 always be set to NEW?

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag; B1: if (r1 ≠ SET) goto L1; L2: Load r2 = data;	/* Initially, data = 0 & flag ≠ SET */ /* L1 & B1 may repeat many times */

Table 3.2: One possible execution of program in Table 3.1

Cycle	Core C1	Core C2	Coherence State of Data	Coherence State of Flag
1	S2: Store flag = SET		Read-only for C2	Read-write for C1
2		L1: Load r1=flag	Read-only for C2	Read-only for C2
3		L2: Load r2=data	Read-only for C2	Read-only for C2
4	S1: Store data = NEW		Read-write for C1	Read-only for C2

describes a few of the ways in which hardware might reorder memory accesses, including these stores. Readers who are not hardware experts may wish to trust that such reordering can happen (e.g., with a write buffer that is not first-in-first-out).

With the reordering of S1 and S2, the execution order may be S2, L1, L2, S1, as illustrated in Table 3.2.

Sidebar: How a Core Might Reorder Memory Access

This sidebar describes a few of the ways in which modern cores may reorder memory accesses to different addresses. Those unfamiliar with these hardware concepts may wish to skip this on first reading. Modern cores may reorder many memory accesses, but it suffices to reason about reordering two memory operations. In most cases, we need to reason only about a core reordering two memory operations to two different addresses, as the sequential execution (i.e., von Neumann) model generally requires that operations to the same address execute in the original program order. We break the possible reorderings down into three cases based on whether the reordered memory operations are loads or stores.

Store-store reordering. Two stores may be reordered if a core has a non-FIFO write buffer that lets stores depart in a different order than the order in which they entered. This might occur if the first store misses in the cache while the second hits or if the second store

can coalesce with an earlier store (i.e., before the first store). Note that these reorderings are possible even if the core executes all instructions in program order. Reordering stores to different memory addresses has no effect on a single-threaded execution. However, in the multithreaded example of Table 3.1, reordering Core C1's stores allows Core C2 to see flag as SET before it sees the store to data. Note that the problem is not fixed even if the write buffer drains into a perfectly coherent memory hierarchy. Coherence will make all caches invisible, but the stores are already reordered.

Load-load reordering. Modern dynamically scheduled cores may execute instructions out of program order. In the example of Table 3.1, Core C2 could execute loads L1 and L2 out of order. Considering only a single-threaded execution, this reordering seems safe because L1 and L2 are to different addresses. However, reordering Core C2's loads behaves the same as reordering Core C1's stores; if the memory references execute in the order L2, S1, S2, and L1, then r2 is assigned 0. This scenario is even more plausible if the branch statement B1 is elided, so no control dependence separates L1 and L2.

Load-store and store-load reordering. Out-of-order cores may also reorder loads and stores (to different addresses) from the same thread. Reordering an earlier load with a later store (a load-store reordering) can cause many incorrect behaviors, such as loading a value after releasing the lock that protects it (if the store is the unlock operation). The example in Table 3.3 illustrates the effect of reordering an earlier store with a later load (a store-load reordering). Reordering Core C1's accesses S1 and L1 and Core C2's accesses S2 and L2 allows the counterintuitive result that both r1 and r2 are 0. Note that store-load reorderings may also arise due to local bypassing in the commonly implemented FIFO write buffer, even with a core that executes all instructions in program order.

A reader might assume that hardware should not permit some or all of these behaviors, but without a better understanding of what behaviors are allowed, it is hard to determine a list of what hardware can and cannot do.

Table 3.3: Can both r1 and r2 be set to 0?

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */

This execution satisfies coherence because the SWMR property is not violated, so incoherence is not the underlying cause of this seemingly erroneous execution result.

20 3. MEMORY CONSISTENCY MOTIVATION AND SEQUENTIAL CONSISTENCY

Let us consider another important example inspired by Dekker's Algorithm for ensuring mutual exclusion, as depicted in Table 3.3. After execution, what values are allowed in $r1$ and $r2$? Intuitively, one might expect that there are three possibilities:

- $(r1, r2) = (0, \text{NEW})$ for execution $S1, L1, S2$, then $L2$
- $(r1, r2) = (\text{NEW}, 0)$ for $S2, L2, S1$, and $L1$
- $(r1, r2) = (\text{NEW}, \text{NEW})$, e.g., for $S1, S2, L1$, and $L2$

Surprisingly, most real hardware, e.g., x86 systems from Intel and AMD, also allows $(r1, r2) = (0, 0)$ because it uses first-in-first-out (FIFO) write buffers to enhance performance. As with the example in Table 3.1, all of these executions satisfy cache coherence, even $(r1, r2) = (0, 0)$.

Some readers might object to this example because it is non-deterministic (multiple outcomes are allowed) and may be a confusing programming idiom. However, in the first place, all current multiprocessors are non-deterministic by default; all architectures of which we are aware permit multiple possible interleavings of the executions of concurrent threads. The illusion of determinism is sometimes, but not always, created by software with appropriate synchronization idioms. Thus, we must consider non-determinism when defining shared memory behavior.

Furthermore, memory behavior is usually defined for *all* executions of all programs, even those that are incorrect or intentionally subtle (e.g., for non-blocking synchronization algorithms). In Chapter 5, however, we will see some high-level language models that allow *some* executions to have undefined behavior, e.g., executions of programs with data races.

3.2 WHAT IS A MEMORY CONSISTENCY MODEL?

The examples in the previous sub-section illustrate that shared memory behavior is subtle, giving value to precisely defining (a) what behaviors programmers can expect and (b) what optimizations system implementors may use. A memory consistency model disambiguates these issues.

A *memory consistency model*, or, more simply, a *memory model*, is a specification of the allowed behavior of multithreaded programs executing with shared memory. For a multithreaded program executing with specific input data, it specifies what values dynamic loads may return. Unlike a single-threaded execution, multiple correct behaviors are usually allowed.

In general, a memory consistency model MC gives rules that partition executions into those obeying MC (*MC executions*) and those disobeying MC (*non-MC executions*). This partitioning of executions, in turn, partitions implementations. An *MC implementation* is a system that permits only MC executions, while a *non-MC implementation* sometimes permits non- MC executions.

Finally, we have been vague regarding the level of programming. We begin by assuming that programs are executables in a hardware instruction set architecture, and we assume that

memory accesses are to memory locations identified by physical addresses (i.e., we are not considering the impact of virtual memory and address translation). In Chapter 5, we will discuss issues with high-level languages (HLLs). We will see then, for example, that a compiler allocating a variable to a register can affect an HLL memory model in a manner similar to hardware reordering memory references.

3.3 CONSISTENCY VS. COHERENCE

Chapter 2 defined cache coherence with two invariants that we informally repeat here. The SWMR invariant ensures that at any time for a memory location with a given address, either (a) one core may write (and read) the address or (b) zero or more cores may only read it. The *Data-Value Invariant* ensures that updates to the memory location are passed correctly so that cached copies of the memory location always contain the most recent version.

It may seem that cache coherence defines shared memory behavior. It does not. As we can see from Figure 3.1, the coherence protocol simply provides the processor core pipeline an abstraction of a memory system. It alone cannot determine shared memory behavior; the

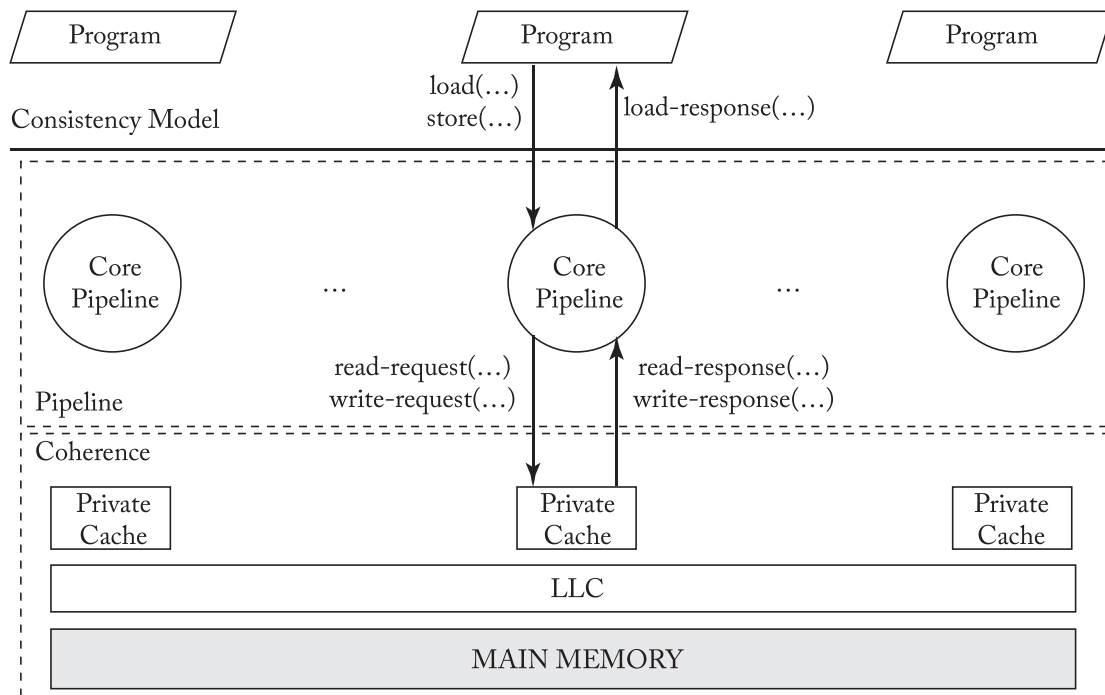


Figure 3.1: A consistency model is enforced by the processor core pipeline combined with the coherence protocol.

22 3. MEMORY CONSISTENCY MOTIVATION AND SEQUENTIAL CONSISTENCY

pipeline matters, too. If, for example, the pipeline reorders and presents memory operations to the coherence protocol in an order contrary to program order—even if the coherence protocol does its job correctly—shared memory correctness may not ensue.

In summary:

- Cache coherence does not equal memory consistency.
- A memory consistency implementation can use cache coherence as a useful “black box.”

3.4 BASIC IDEA OF SEQUENTIAL CONSISTENCY (SC)

Arguably the most intuitive memory consistency model is SC. It was first formalized by Lamport [12], who called a single processor (core) *sequential* if “the result of an execution is the same as if the operations had been executed in the order specified by the program.” He then called a multiprocessor *sequentially consistent* if “the result of any execution is the same as if the operations of all processors (cores) were executed in some sequential order, and the operations of each individual processor (core) appear in this sequence in the order specified by its program.” This total order of operations is called *memory order*. In SC, memory order respects each core’s program order, but other consistency models may permit memory orders that do not always respect the program orders.

Figure 3.2 depicts an execution of the example program from Table 3.1. The middle vertical downward arrow represents the memory order ($\langle m \rangle$) while each core’s downward arrow represents its program order ($\langle p \rangle$). We denote memory order using the operator $\langle m \rangle$, so $op1 \langle m \rangle$

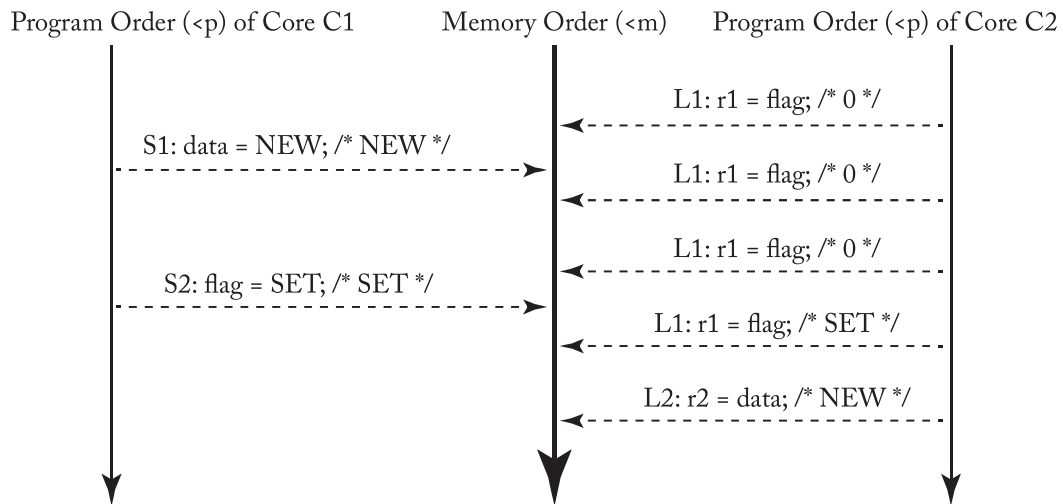


Figure 3.2: A sequentially consistent execution of Table 3.1’s program.

op2 implies that op1 precedes op2 in memory order. Similarly, we use the operator $\prec_p$ to denote program order for a given core, so $\text{op1} \prec_p \text{op2}$ implies that op1 precedes op2 in that core's program order. Under SC, memory order *respects* each core's program order. "Respects" means that $\text{op1} \prec_p \text{op2}$ implies $\text{op1} \prec_m \text{op2}$. The values in comments (*/* ... */*) give the value loaded or stored. This execution terminates with r2 being NEW. More generally, all executions of Table 3.1's program terminate with r2 as NEW. The only non-determinism—how many times L1 loads flag as 0 before it loads the value SET once—is unimportant.

This example illustrates the value of SC. In Section 3.1, if you expected that r2 must be NEW, you were perhaps independently inventing SC, albeit less precisely than Lamport.

The value of SC is further revealed in Figure 3.3, which illustrates four executions of the program from Table 3.3. Figure 3.3a–c depict SC executions that correspond to the three intuitive outputs: $(r1, r2) = (0, \text{NEW})$, $(\text{NEW}, 0)$, or (NEW, NEW) . Note that Figure 3.3c depicts only one of the four possible SC executions that leads to $(r1, r2) = (\text{NEW}, \text{NEW})$; this execution is $\{S1, S2, L1, L2\}$, and the others are $\{S1, S2, L2, L1\}$, $\{S2, S1, L1, L2\}$, and $\{S2, S1, L2, L1\}$. Thus, across Figure 3.3a–c, there are six legal SC executions.

Figure 3.3d shows a non-SC execution corresponding to the output $(r1, r2) = (0, 0)$. For this output, there is no way to create a memory order that respects program orders. Program order dictates that:

- $S1 \prec_p L1$
- $S2 \prec_p L2$

But memory order dictates that:

- $L1 \prec_m S2$ (so r1 is 0)
- $L2 \prec_m S1$ (so r2 is 0)

Honoring all these constraints results in a cycle, which is inconsistent with a total order. The extra arcs in Figure 3.3d illustrate the cycle.

We have just seen six SC executions and one non-SC execution. This can help us understand SC implementations: an SC implementation must allow one or more of the first six executions, but cannot allow the seventh execution.

3.5 A LITTLE SC FORMALISM

In this section, we define SC more precisely, especially to allow us to compare SC with the weaker consistency models in the next two chapters. We adopt the formalism of Weaver and Germond [20]—an axiomatic method to specify consistency which we discuss more in Chapter 11—with the following notation: $L(a)$ and $S(a)$ represent a load and a store, respectively, to address a . Orders $\prec_p$ and $\prec_m$ define program and global memory order, respectively. Program

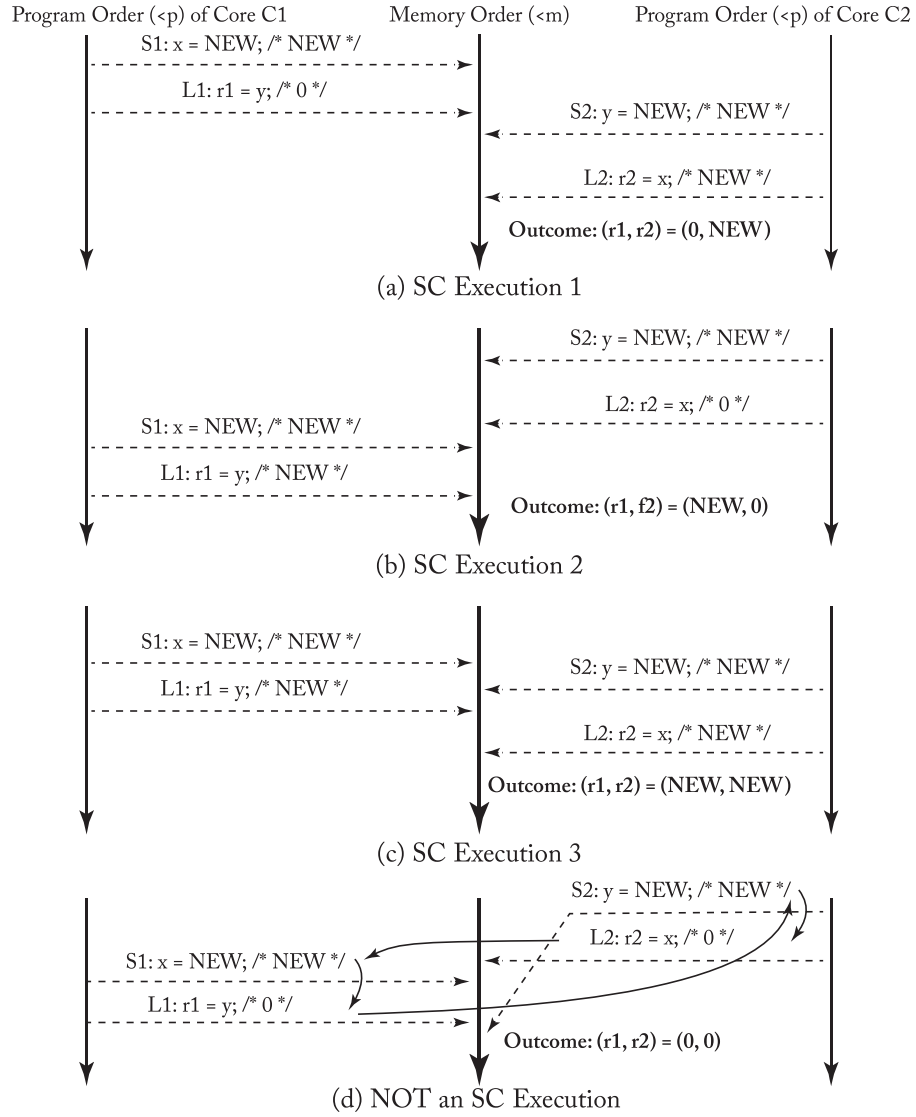


Figure 3.3: Four alternative executions of Table 3.3's program.

order $\prec_p$ is a per-core total order that captures the order in which each core logically (sequentially) executes memory operations. Global memory order $\prec_m$ is a total order on the memory operations of all cores.

An **SC execution** requires the following.

- (1) All cores insert their loads and stores into the order $\prec_m$ respecting their program order, regardless of whether they are to the same or different addresses (i.e., $a=b$ or $a \neq b$). There are four cases:
 - If $L(a) \prec_p L(b) \Rightarrow L(a) \prec_m L(b)$ /* Load $\rightarrow$ Load */
 - If $L(a) \prec_p S(b) \Rightarrow L(a) \prec_m S(b)$ /* Load $\rightarrow$ Store */
 - If $S(a) \prec_p S(b) \Rightarrow S(a) \prec_m S(b)$ /* Store $\rightarrow$ Store */
 - If $S(a) \prec_p L(b) \Rightarrow S(a) \prec_m L(b)$ /* Store $\rightarrow$ Load */
- (2) Every load gets its value from the last store before it (in global memory order) to the same address:

Value of $L(a)$ = Value of $\text{MAX}_{\prec_m} \{S(a) \mid S(a) \prec_m L(a)\}$, where $\text{MAX}_{\prec_m}$ denotes “latest in memory order.”

Atomic read-modify-write (RMW) instructions, which we discuss in more depth in Section 3.9, further constrain allowed executions. Each execution of a test-and-set instruction, for example, requires that the load for the test and the store for the set logically appear consecutively in the memory order (i.e., no other memory operations for the same or different addresses interpose between them).

We summarize SC’s ordering requirements in Table 3.4. The table specifies which program orderings are enforced by the consistency model. For example, if a given thread has a load before a store in program order (i.e., the load is “Operation 1” and the store is “Operation 2” in the table), then the table entry at this intersection is an “X” which denotes that these operations must be performed in program order. For SC, all memory operations must appear to perform in program order; under other consistency models, which we study in the next two chapters, some of these ordering constraints are relaxed (i.e., some entries in their ordering tables do not contain an “X”).

An **SC implementation** permits only SC executions. Strictly speaking, this is the *safety* property for SC implementations (do no harm). SC implementations should also have some *liveness* properties (do some good). Specifically, a store must become eventually visible to a load that is repeatedly attempting to load that location. This property, referred to as eventual write-propagation, is typically ensured by the coherence protocol. More generally, starvation avoidance and some fairness are also valuable, but these issues are beyond the scope of this discussion.

Table 3.4: SC ordering rules. An “X” denotes an enforced ordering.

Operation 1	Operation 2			
		Load	Store	RMW
Load		X	X	X
Store		X	X	X
RMW		X	X	X

3.6 NAIVE SC IMPLEMENTATIONS

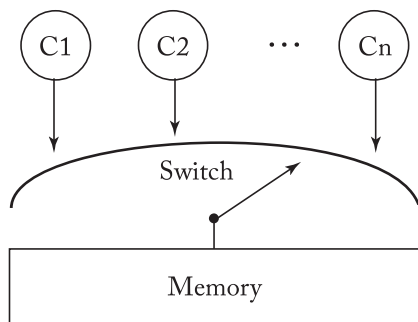
SC permits two naive implementations that make it easier to understand which executions SC permits.

The Multitasking Uniprocessor

First, one can implement SC for multithreaded user-level software by executing all threads on a single sequential core (a uniprocessor). Thread T1’s instructions execute on core C1 until a context switch to thread T2, etc. On a context switch, any pending memory operations must be completed before switching to the new thread. Because each thread’s instructions in its quanta execute as one atomic block (and because the uniprocessor correctly honors memory dependencies), all of the SC rules are enforced.

The Switch

Second, one can implement SC with a set of cores C, a single switch, and memory, as depicted in Figure 3.4. Assume that each core presents memory operations to the switch one at a time in its program order. Each core can use any optimizations that do not affect the order in which it presents memory operations to the switch. For example, a simple five-stage in-order pipeline with branch prediction can be used.



Each core C_i seeks to do its next memory access in its program order $\langle p \rangle$.

The switch selects one core, allows it to complete one memory access, and repeats; this defines memory order $\langle m \rangle$.

Figure 3.4: A simple SC implementation using a memory switch.

Assume next that the switch picks one core, allows memory to fully satisfy the load or store, and repeats this process as long as requests exist. The switch may pick cores by any method (e.g., random) that does not starve a core with a ready request. This implementation operationally implements SC by construction.

Assessment

The good news from these implementations is that they provide operational models defining (1) allowed SC executions and (2) SC implementation “gold standards.” (In Chapter 11, we will see that such operational models can be used to formally specify consistency models.) The switch implementation also serves as an existence proof that SC can be implemented without caches or coherence.

The bad news, of course, is that the performance of these implementations does not scale up with increasing core count, due to the sequential bottleneck of using a single core in the first case and the single switch/memory in the second case. These bottlenecks have led some people to incorrectly conclude that SC precludes true parallel execution. It does not, as we will see next.

3.7 A BASIC SC IMPLEMENTATION WITH CACHE COHERENCE

Cache coherence facilitates SC implementations that can execute *non-conflicting* loads and stores—two operations *conflict* if they are to the same address and at least one of them is a store—completely in parallel. Moreover, creating such a system is conceptually simple.

Here, we treat coherence as mostly a black box that implements the SWMR invariant of Chapter 2. We provide some implementation intuition by opening the coherence block box slightly to reveal simple level-one (L1) caches that:

- use state *modified* (*M*) to denote an L1 block that one core can write and read,
- use state *shared* (*S*) to denote an L1 block that one or more cores can only read, and
- have *GetM* and *GetS* denote coherence requests to obtain a block in *M* and *S*, respectively.

We do not require a deep understanding of how coherence is implemented, as discussed in Chapter 6 and beyond.

Figure 3.5a depicts the model of Figure 3.4 with the switch and memory replaced by a cache-coherent memory system represented as a black box. Each core presents memory operations to the cache-coherent memory system one at a time in its program order. The memory system fully satisfies each request before beginning the next request for the same core.

Figure 3.5b “opens” the memory system black box a little to reveal that each core connects to its own L1 cache (we will talk about multithreading later). The memory system can respond to a load or store to block *B* if it has *B* with appropriate coherence permissions (state *M* or *S* for loads and *M* for stores). Moreover, the memory system can respond to requests from different

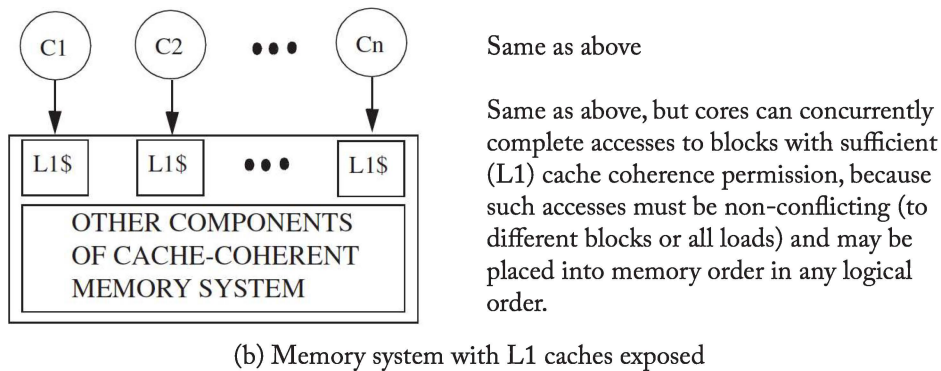
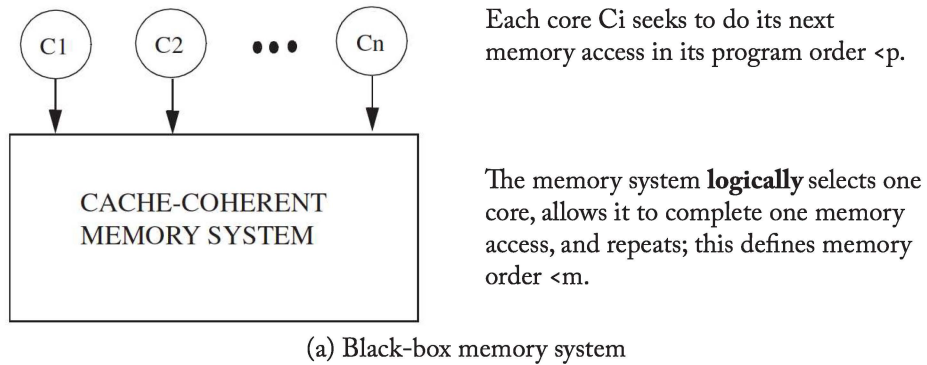


Figure 3.5: Implementing SC with cache coherence.

cores in parallel, provided that the corresponding L1 caches have the appropriate permissions. For example, Figure 3.6a depicts the cache states before four cores each seek to do a memory operation. The four operations do not conflict, can be satisfied by their respective L1 caches, and therefore can be done concurrently. As depicted in Figure 3.6b, we can arbitrarily order these operations to obtain a legal SC execution model. More generally, operations that can be satisfied by L1 caches always can be done concurrently because coherence's SWMR invariant ensures they are non-conflicting.

Assessment

We have created an implementation of SC that:

- fully exploits the latency and bandwidth benefits of caches,
- is as scalable as the cache coherence protocol it uses, and
- decouples the complexities of implementing cores from implementing coherence.

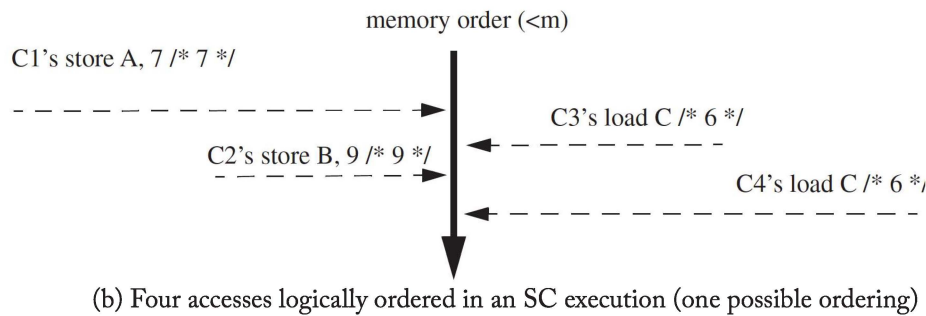
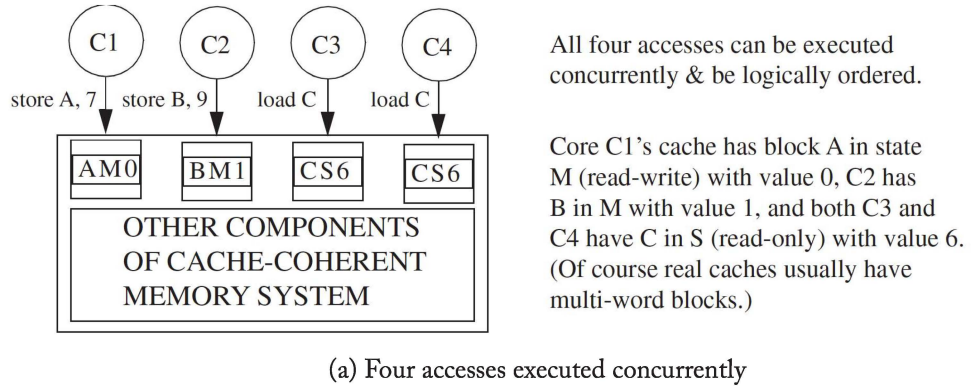


Figure 3.6: A concurrent SC execution with cache coherence.

3.8 OPTIMIZED SC IMPLEMENTATIONS WITH CACHE COHERENCE

Most real core implementations are more complicated than our basic SC implementation with cache coherence. Cores employ features like prefetching, speculative execution, and multithreading in order to improve performance and tolerate memory access latencies. These features interact with the memory interface, and we now discuss how these features impact the implementation of SC. It is worth bearing in mind that any feature or optimization is legal as long as it does not produce an end result (values returned by loads) that violates SC.

Non-Binding Prefetching

A non-binding prefetch for block B is a request to the coherent memory system to change B's coherence state in one or more caches. Most commonly, prefetches are requested by software, core hardware, or the cache hardware to change B's state in the level-one cache to permit loads (e.g., B's state is M or S) or loads and stores (B's state is M) by issuing coherence requests such

30 3. MEMORY CONSISTENCY MOTIVATION AND SEQUENTIAL CONSISTENCY

as GetS and GetM. Importantly, in no case does a non-binding prefetch change the state of a register or data in block B. The effect of the non-binding prefetch is limited to within the “cache-coherent memory system” block of Figure 3.5a, making the effect of non-binding prefetches on the memory consistency model to be the functional equivalent of a no-op. So long as the loads and stores are performed in program order, it does not matter in what order coherence permissions are obtained.

Implementations may do non-binding prefetches without affecting the memory consistency model. This is useful for both internal cache prefetching (e.g., stream buffers) and more aggressive cores.

Speculative Cores

Consider a core that executes instructions in program order, but also does branch prediction wherein subsequent instructions, including loads and stores, begin execution, but may be squashed (i.e., have their effects nullified) on a branch misprediction. These squashed loads and stores can be made to look like non-binding prefetches, enabling this speculation to be correct because it has no effect on SC. A load after a branch prediction can be presented to the L1 cache, wherein it either misses (causing a non-binding GetS prefetch) or hits and then returns a value to a register. If the load is squashed, the core discards the register update, erasing any functional effect from the load—as if it never happened. The cache does not undo non-binding prefetches, as doing so is not necessary and prefetching the block can help performance if the load gets re-executed. For stores, the core may issue a non-binding GetM prefetch early, but it does not present the store to the cache until the store is guaranteed to commit.

Flashback to Quiz Question 1: In a system that maintains sequential consistency, a core must issue coherence requests in program order. *True or false?*

Answer: *False!* A core may issue coherence requests in any order.

Dynamically Scheduled Cores

Many modern cores dynamically schedule instruction execution out of program order to achieve greater performance than statically scheduled cores that must execute instructions in strict program order. A single-core processor that uses dynamic or out-of-(program-)order scheduling must simply enforce true data dependences within the program. However, in the context of a multicore processor, dynamic scheduling introduces a new issue: memory consistency speculation. Consider a core that wishes to dynamically reorder the execution of two loads, L1 and L2 (e.g., because L2’s address is computed before L1’s address). Many cores will speculatively execute L2 before L1, and they are predicting that this reordering is not visible to other cores, which would violate SC.

Speculating on SC requires that the core verify that the prediction is correct. Gharachorloo et al. [8] presented two techniques for performing this check. First, after the core speculatively

executes L2, but before it commits L2, the core could check that the speculatively accessed block has not left the cache. So long as the block remains in the cache, its value could not have changed between the load's execution and its commit. To perform this check, the core tracks the address loaded by L2 and compares it to blocks evicted and to incoming coherence requests. An incoming GetM indicates that another core could observe L2 out of order, and this GetM would imply a mis-speculation and squash the speculative execution.

The second checking technique is to replay each speculative load when the core is ready to commit the load² [2, 17]. If the value loaded at commit does not equal the value that was previously loaded speculatively, then the prediction was incorrect. In the example, if the replayed load value of L2 is not the same as the originally loaded value of L2, then the load-load reordering has resulted in an observably different execution and the speculative execution must be squashed.

Non-Binding Prefetching in Dynamically Scheduled Cores

A dynamically scheduled core is likely to encounter load and store misses out of program order. For example, assume that program order is Load A, Store B, then Store C. The core may initiate non-binding prefetches “out of order,” e.g., GetM C first and then GetS A and GetM B in parallel. SC is not affected by the order of non-binding prefetches. SC requires only that a core's loads and stores (appear to) access its level-one cache in program order. Coherence requires the level-one cache blocks to be in the appropriate states to receive loads and stores.

Importantly, SC (or any other memory consistency model):

- dictates the order in which loads and stores (appear to) get applied to coherent memory but
- does NOT dictate the order of coherence activity.

Flashback to Quiz Question 2: The memory consistency model specifies the legal orderings of coherence transactions. *True or false?*

Answer: *False!*

Multithreading

Multithreading—at coarse grain, fine grain, or simultaneous—can be accommodated by SC implementations. Each multithreaded core should be made logically equivalent to multiple (virtual) cores sharing each level-one cache via a switch where the cache chooses which virtual core to service next. Moreover, each cache can actually serve multiple non-conflicting requests concurrently because it can pretend that they were serviced in some order. One challenge is ensuring that a thread T1 cannot read a value written by another thread T2 on the same core before the store has been made “visible” to threads on other cores. Thus, while thread T1 may read the

²Roth [17] demonstrated a scheme for avoiding many load replays by determining when they are not necessary.

32 3. MEMORY CONSISTENCY MOTIVATION AND SEQUENTIAL CONSISTENCY

value as soon as thread T2 inserts the store in the memory order (e.g., by writing it to a cache block in state M), it cannot read the value from a shared load-store queue in the processor core.

Sidebar: Advanced SC Optimizations

This sidebar describes some advanced SC optimizations.

Post-retirement speculation. A single-core processor typically employs a structure called the write (store) buffer for hiding the latency of store misses; a store retires from the processor pipeline into the write buffer from where it drains into the cache/memory system off the critical path. This is safe on a single-core as long as loads check the write buffer for outstanding stores to the same address. On a multicore, however, the SC ordering rules preclude the naive use of a write buffer. Dynamically scheduled cores can hide some, but not all, of the store miss latency. To hide even more of the store miss latency, there have been many proposals for aggressive implementations of SC, utilizing speculation beyond the instruction window. The key idea is to speculatively retire loads and stores past pending store misses, while maintaining the state of speculatively retired instructions separately at either a fine granularity [9, 16] or coarse-grained chunks [1, 3, 11, 19].

Non-speculative reordering. It is even possible to non-speculatively perform memory operations out of order while enforcing SC, as long as the reordering is invisible to other cores [7, 18]. How do you ensure that reordering is invisible to other cores, without rollback recovery?

One approach (dubbed *coherence delaying*) involves delaying coherence requests: specifically, when a younger memory operation is retired past a pending older one, coherence requests to the younger one's location are delayed until the older memory operation retires. There is an inherent deadlock hazard with coherence delaying that necessitates careful deadlock avoidance mechanisms. In the example shown in Table 3.3, if both loads L1 and L2 retire past the stores and coherence requests to the respective locations are delayed, this can prevent either of the stores from completing, thereby leading to a deadlock.

Another approach (dubbed *predecessor serialization*) requires the older memory operation to do just enough—typically serializing at a central point—to ensure that it is safe for younger operations to complete past it. Conflict ordering [6] allows for loads and stores to retire past a pending store miss, as soon as the pending store serializes at the directory and determines a global list of pending stores; as long as the younger memory operation does not conflict with this list, it can safely retire. Gope and Lipasti [4] propose an approach tailored for in-order processors, wherein every load or store obtains a mutex from the directory in program order, but can retire out of order.

Finally, it is possible to leverage the help of the compiler or the memory management unit to determine accesses that can be safely reordered [5]. For example, two accesses to thread-private or read-only variables can be safely reordered.

3.9 ATOMIC OPERATIONS WITH SC

To write multithreaded code, a programmer needs to be able to synchronize the threads, and such synchronization often involves atomically performing pairs of operations. This functionality is provided by instructions that atomically perform a “read-modify-write” (RMW), such as the well-known “test-and-set,” “fetch-and-increment,” and “compare-and-swap.” These atomic instructions are critical for proper synchronization and are used to implement spin-locks and other synchronization primitives. For a spin-lock, a programmer might use an RMW to atomically read whether the lock’s value is unlocked (e.g., equal to 0) and write the locked value (e.g., equal to 1). For the RMW to be atomic, the read (load) and write (store) operations of the RMW must appear consecutively in the total order of operations required by SC.

Implementing atomic instructions in the microarchitecture is conceptually straightforward, but naive designs can lead to poor performance for atomic instructions. A correct but simplistic approach to implementing atomic instructions would be for the core to effectively lock the memory system (i.e., prevent other cores from issuing memory accesses) and perform its read, modify, and write operations to memory. This implementation, although correct and intuitive, sacrifices performance.

More aggressive implementations of RMWs leverage the insight that SC requires only the appearance of a total order of all requests. Thus, an atomic RMW can be implemented by first having a core obtain the block in state M in its cache, if the block is not already there in that state. The core then needs to only load and store the block in its cache—without any coherence messages or bus locking—as long as it waits to service any incoming coherence request for the block until after the store. This waiting does not risk deadlock because the store is guaranteed to complete.

Flashback to Quiz Question 3: To perform an atomic read-modify-write instruction (e.g., test-and-set), a core must always communicate with the other cores. *True or false?*

Answer: *False!*

An even more optimized implementation of RMWs could allow more time between when the load part and store part perform, without violating atomicity. Consider the case where the block is in a read-only state in the cache. The load part of the RMW can speculatively perform immediately, while the cache controller issues a coherence request to upgrade the block’s state to read-write. When the block is then obtained in read-write state, the write part of the RMW performs. As long as the core can maintain the illusion of atomicity, this implementation is correct. To check whether the illusion of atomicity is maintained, the core must check whether the loaded block is evicted from the cache between the load part and the store part; this speculation support is the same as that needed for detecting mis-speculation in SC (Section 3.8).

3.10 PUTTING IT ALL TOGETHER: MIPS R10000

The MIPS R10000 [21] provides a venerable, but clean, commercial example for a speculative microprocessor that implements SC in cooperation with a cache-coherent memory hierarchy. Herein, we concentrate on aspects of the R10000 that pertain to implementing memory consistency.

The R10000 is a four-way superscalar RISC processor core with branch prediction and out-of-order execution. The chip supports writeback caches for L1 instructions and L1 data, as well as a private interface to an (off-chip) unified L2 cache.

The chip's main *system interface bus* supports cache coherence for up to four processors, as depicted in Figure 3.7 (adapted from Figure 1 in Yeager [21]). To construct an R10000-based system with more processors, such as the SGI Origin 2000 (discussed at length in Section 8.8.1), architects implemented a directory coherence protocol that connects R10000 processors via the system interface bus and a specialized Hub chip. In both cases, the R10000 processor core sees a coherent memory system that happens to be partially on-chip and partially off-chip.

During execution, an R10000 core issues (speculative) loads and stores in program order into an *address queue*. A load obtains a (speculative) value from the last store before it to the same address or, if none, the data cache. Loads and stores commit in program order and then remove their address queue entries. To commit a store, the L1 cache must hold the block in state M and the store's value must be written atomically with the commit.

Importantly, the eviction of a cache block—due to a coherence invalidation or to make room for another block—that contains a load's address in the address queue squashes the load and all subsequent instructions, which then re-execute. Thus, when a load finally commits, the loaded block was continuously in the cache between when it executed and when it commits, so it must get the same value as if it executed at commit. Because stores actually write to the cache at commit, the R10000 logically presents loads and stores in program order to the coherent memory system, thereby implementing SC, as discussed earlier.

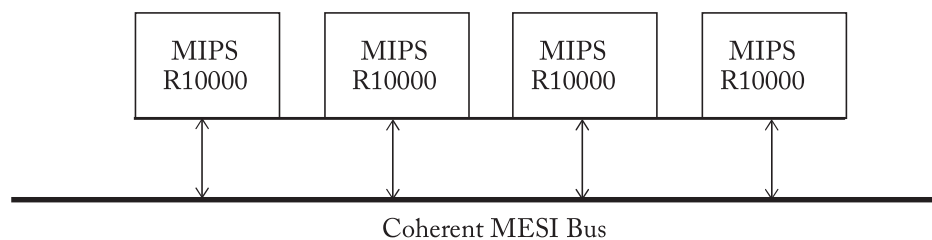


Figure 3.7: Coherent MESI bus connects up to four MIPS R10000 processors.

3.11 FURTHER READING REGARDING SC

Below we highlight a few of the papers from the vast literature surrounding SC.

Lamport [12] defined SC. As far as we know, Meixner and Sorin [15] were the first to prove that a system in which cores present loads and stores in program order to a cache coherent memory system was sufficient to implement SC, even as this result was intuitively believed for some time.

SC can be compared with database serializability [10]. The two concepts are similar in that they both insist that the operations from all entities appear to affect shared state in a serial order. The concepts differ due to the nature of and expectation for operations and shared state. With SC, each operation is a single memory access to volatile state (memory) that is assumed not to fail. With serializability, each operation is a transaction on a database that can read and write multiple database entities and is expected to obey ACID properties: Atomic—all or nothing even with failures; Consistent—leave the database consistent; Isolated—no effect from concurrent transactions; and Durable—effects survive crashes and power loss.

We followed Lamport and SPARC to define a total order of all memory accesses. While this can ease intuition for some, it is not necessary. Recall that two accesses *conflict* if they are from different threads, access the same location, and at least one is a store (or RMW). Instead of a total order, one can just define the constraints on conflicting accesses and leave non-conflicting accesses unordered, as pioneered by Shasha and Snir [18]. This view can be especially valuable for the relaxed models of Chapter 5.

Finally, a cautionary tale. We stated earlier (Section 3.7) that one way to check whether a speculatively executed load could have been observed out of order is to remember the value *A* that is speculatively read by a load and to commit the load if, at commit, the memory location has the same value *A*. Martin et al. [14] show that this is *not* the case for cores that perform value prediction [13]. With value prediction, when a load executes, the core can speculate on its value. Consider a core that speculates that a load of block *X* will produce the value *A*, although the value is actually *B*. Between when the core speculates on the load of *X* and when it replays the load at commit, another core changes block *X*'s value to *A*. The core then replays the load at commit, compares the two values, which are equal, and mistakenly determines that the speculation was correct. The system can violate SC if it speculates in this way. This situation is analogous to the so-called ABA problem (http://en.wikipedia.org/wiki/ABA_problem), and Martin et al. showed that there are ways of checking speculation in the presence of value prediction that avoid the possibility of consistency violations (e.g., by also replaying all loads dependent on the initially speculated load). The point of this discussion is not to delve into the details of this particular corner case or its solutions, but rather to convince you to prove that your implementation is correct rather than rely on intuition.

3.12 REFERENCES

- [1] C. Blundell, M. M. K. Martin, and T. F. Wenisch. InvisiFence: Performance-transparent memory ordering in conventional multiprocessors. In *Proc. of the 36th Annual International Symposium on Computer Architecture*, June 2009. DOI: [10.1145/1555754.1555785](https://doi.org/10.1145/1555754.1555785). 32
- [2] H. W. Cain and M. H. Lipasti. Memory ordering: A value-based approach. In *Proc. of the 31st Annual International Symposium on Computer Architecture*, June 2004. DOI: [10.1109/isca.2004.1310766](https://doi.org/10.1109/isca.2004.1310766). 31
- [3] L. Ceze, J. Tuck, P. Montesinos, and J. Torrellas. BulkSC: Bulk enforcement of sequential consistency. In *Proc. of the 34th Annual International Symposium on Computer Architecture*, June 2007. DOI: [10.1145/1250662.1250697](https://doi.org/10.1145/1250662.1250697). 32
- [4] D. Gope and M. H. Lipasti. Atomic SC for simple in-order processors. In *20th IEEE International Symposium on High Performance Computer Architecture*, 2014. DOI: [10.1109/hpca.2014.6835950](https://doi.org/10.1109/hpca.2014.6835950). 32
- [5] A. Singh, S. Narayanasamy, D. Marino, T.D Millstein, and M. Musuvathi. End-to-end sequential consistency. In *39th International Symposium on Computer Architecture*, 2012. DOI: [10.1109/isca.2012.6237045](https://doi.org/10.1109/isca.2012.6237045). 32
- [6] C. Lin, V. Nagarajan, R. Gupta, and B. Rajaram. Efficient sequential consistency via conflict ordering. In *Proc. of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems ASPLOS*, 2012. DOI: [10.1145/2150976.2151006](https://doi.org/10.1145/2150976.2151006). 32
- [7] K. Gharachorloo, S. V. Adve, A. Gupta, J. Hennessy, and M. D. Hill. Specifying system requirements for memory consistency models. *Technical Report CSL-TR93-594*, Stanford University, December 1993. 32
- [8] K. Gharachorloo, A. Gupta, and J. Hennessy. Two techniques to enhance the performance of memory consistency models. In *Proc. of the International Conference on Parallel Processing*, vol. I, pp. 355–64, August 1991. 30
- [9] C. Guiady, B. Falsafi, and T. Vijaykumar. Is SC + ILP = RC? In *Proc. of the 26th Annual International Symposium on Computer Architecture*, pp. 162–71, May 1999. DOI: [10.1109/isca.1999.765948](https://doi.org/10.1109/isca.1999.765948). 32
- [10] J. Gray and A. Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann Publishers, 1993. 35
- [11] L. Hammond et al. Transactional memory coherence and consistency. In *Proc. of the 31st Annual International Symposium on Computer Architecture*, June 2004. DOI: [10.1109/isca.2004.1310767](https://doi.org/10.1109/isca.2004.1310767). 32